lambdaSearch

Prosjektrapport

INF221 Avansert funksjonell programmering

Martin Elde Knutsen

29. 04 2026

1 Prosjekt description

1.1 What was the initial idea / background of the project?

The reason I wanted to make this project is that I really like CLI tools. I use them a lot every day. But I think some of them have unnecessary, bad, and unintuitive syntax. That's why I wanted to make my own tool where I had complete control over the syntax. I also wanted to build a TUI around it, mostly because brick sounded like a fun library to try out. So the idea was to use FFI, MegaParsec, and brick to make a CLI tool for searching for files on my PC. I Also wanted to try to use STM an concurrency to search through the files faster.

1.2 Project goals

- Make the searching program fast, efficient and safe.
- Call C functions to retrieve file metadata, with a little as possible overhead.
- Use the Either monad for error handling.
- Use the STM monad for concurrency.
- Parse CLI input with MegaParsec.
- Parse a configuration file.
- Write good readable code

1.3 Result

- I managed to make the CLI with the syntax that I wanted. But I did not implement concurrency, because it was already faster than the built-in find command and I wanted to prioritise testing and the TUI.

1.3.1 Implemented

- Core directory traversal (filter by regex, extension, hidden, ignored).
- Command execution (-x flag substituting {}).
- TUI with brick (live search, vim-bindings, open in $EDITOR).
- C FFI bindings (readdir, etc.) for fast metadata retrieval.
- CLI parsing using megaparsec.
- Error handling with Either/ExceptT for permission issues.
- test suite (Tasty, QuickCheck, HUnit).

1.3.2 Not Implemented

- Concurrency (STM / parallel traversal).
- Configuration file parsing.

1.4 Future extensions

- Implement concurrent searching

2 Description of the functional programming techniques

2.1 Monadic Parsing Combinators

When parsing the for the cli i use of lot of combinators. Making it very readable and elagant .

- Eksample:

```
pFlags :: Parser DataFlags
pFlags = choice
    [ SearchPatternFlag <$> ((string' "-p" <|> string  "--pattern"   ) *> sc *> parseWord)
     , HiddenFilesFlag  <$  ( string' "-a" <|> string  "--show--dots")
       ...
```

- We use the fmap operator (<$>) to lift the parsed result into the DataFlags context.
- Next is the acctual parser. Three things to notice here:

1. The string' function enables case-insensitive parsing (e.g., both -p and -P).
2. I use the <|> operator, which here sort of acts like a or. It tries to parse -p, and if that fails, it tries to parse –pattern. Because the type signature of this operator is f a -> f a -> f a, you can treat the combined expression as a single parser. This means anywhere you parse one thing, you can easily try to parse alternatives without any hassle. Of course, the inner type a must be the same for both parsers.
3. The sequence operator (*>) consumes and discards the flag prefix. We chain this with sc to skip whitespace before capturing the actual argument.

- For HiddenFilesFlag, we use <$. Since this flag takes no arguments, <$ directly replaces the parsed string with the data constructor.
- And its all wrapped in chose which act like a sequence of <|>

2.2 Monad Transformers and Monadic Error Handling

To make the directory traversal as fast as possible, I used the Foreign Function Interface (FFI) to bind directly to C POSIX functions. But to keep the Haskell side safe, I wrapped these impure calls in Monad Transformers to handle error and end of dir exeptions.

- Example:

```
type DirContentT = ExceptT DirError IO (Maybe DirContent)

readDirEnt :: DirStream ->  DirContentT
readDirEnt dir = ExceptT readContent
    where
    readContent :: IO (Either DirError (Maybe DirContent))
    readContent = do
      -- ... raw IO and FFI calls to c_readdir ...
```

- First, I define DirContentT using ExceptT. This layers an exception context (DirError) over the IO monad. This lets me sequence IO actions but still cleanly short-circuit if a specific directory error happens (like a permission denied error).
- Inside readContent, I do the actual raw IO calls to the C function c_readdir. Depending on the Errno returned by C, I can return pure . Right $ Nothing (if we hit the end of the folder) or pure . Left $ ReadDirErr err if it actually failed.

2.3 Higher-Order Functions and Folds

2.3.1 In the parser

- To keep track of which flags the user passed in the CLI, I used a functional fold. This is a clean way to build up the configuration state purely.
- Example:

```
parseFlags :: Parser Arguments
parseFlags = do
  fs <- parseAllFlags
  pure $ foldl' applyFlag emptyFilterFlags fs
```

- Here, parseAllFlags returns a list of parsed DataFlags.
- To process them, I use foldl' (the strict left fold) to iterate over this list. Why strict fold? Beacuse we want to avoid space leaks with accumulating expressions on the heap. It is probably not an issue on my program, but it is just good practice.
- The magic happens with the applyFlag function, which has the type signature Arguments -> DataFlags -> Arguments. It basically acts as a state transition function. It takes the current Arguments state, looks at the new DataFlags we just parsed, and returns a newly updated Arguments record.
- So we start with emptyFilterFlags as our base state, fold over the list of parsed flags, and get our fully constructed configuration.

2.3.2 In the travaveselfunction

First, I just wanted to point out how perfect Haskell is for these kinds of traversal functions it makes so much sense to write these traversal functions recursively. Essentially, the only thing you want to do is apply the same logic to every directory while you search and accumulate the results.

In the main function, where I do the traversals, I use higher-order functions and a generalized fold function.

```
foldDirectoryTree
    :: (a -> RawFilePath -> DirContent -> IO a) -- Foldfunction
    -> a --
    -> RawFilePath
    -> IO a
foldDirectoryTree foldFunc acc rootPath  = do

    isDir <- isDirectory <$> getFileStatus  rootPath

    if not isDir
        then pure acc
        else traverseDirectoryContents innerloop acc rootPath
    where
        innerloop currentAcc dc@(typ,filename) = do

            let filePath = rootPath  </> filename
            let isDir = typ == dtDir
            -- legge funskjonen på
            nextAcc <- foldFunc currentAcc rootPath dc
            if not isDir
                then pure nextAcc
                else foldDirectoryTree foldFunc nextAcc filePath
```

- The first thing to notice is the type signature. It takes a function (a -> RawFilePath -> DirContent -> IO a) which acts as our step function, an initial accumulator of type a, and the starting path.
- Because of this generic signature, foldDirectoryTree doesn't actually know what data it is collecting (it could be a list of files, a count, etc.). It just knows how to walk the tree and thread a state of type a through the execution.
- If the current path is a directory, it delegates to traverseDirectoryContents using a custom innerloop helper function to process each item inside that directory.
- The innerloop is where the recursive happens. It applies the function foldFunc to the current item to compute the nextAcc (our updated state).
- Then, if the current item happens to be another directory, it recursively calls foldDirectoryTree on that new path, passing in the nextAcc. This threads the accumulated state down into deeply nested subdirectories and back up.

So i can use the generalized function to accumulate fileinformation

```
treversRecursively :: Arguments -> [FileInfomation] -> RawFilePath -> IO [FileInfomation]
treversRecursively args = foldDirectoryTree foldFunc
    where
    regexCompiled = compileRegexFilter args
    foldFunc :: [FileInfomation] -> RawFilePath -> DirContent -> IO [FileInfomation]
    foldFunc acc parentPath dc@(typ,file)  = ...
```

Or i can use it to count the files

```
countFiles :: Integer -> RawFilePath -> IO Integer
countFiles  = foldDirectoryTree foldFunc
    where
    foldFunc ::  Integer -> RawFilePath -> DirContent -> IO Integer
    foldFunc s  _ ((== dtDir) -> True ,_) = pure (1 +  s)
    foldFunc s _ _                        = pure s
```

2.4 Algebraic data types

For storing the config of my search logic i used records.

- Eksampel:

```
data Arguments = Arguments {
      regxPattern    :: Maybe SearchPattern
    , exclude        :: Maybe SearchFilters
    , extention      :: Maybe Extention
    , hideHidden     :: Bool
    , applyedCommand :: Maybe ConstrucedCommand
}  deriving (Eq, Show)
```

- The key functional technique is using the Maybe type (like Maybe SearchPattern) to explicitly encode whether a filter is active directly in the type system.
- Instead of relying on null pointers or empty strings, Maybe makes invalid states unrepresentable. An unused filter is Nothing.
- This forces explicit pattern matching (Just val vs Nothing) in my filter functions, providing compile-time guarantees that I won't crash from null references or accidentally apply an empty filter.

```haskell
getRexPattern :: Maybe Regex -> RawFilePath -> Bool
getRexPattern Nothing      _  = True
getRexPattern (Just regex) fp = matchTest regex fp
```

- If where to remove the pattern match on Nothing it will give me a compiler warning

Another exampel when i needed to constuct the the command that i would execute

```haskell
data Args = Text String | PathToSubs
    deriving (Eq,Show)

-- | alias som beskriver en kommaddo eksterne kommandoer og argumenter.
--  String is just the name of the program, eks cat, sed, pandoc
type ConstrucedCommand = (String, [Args])
```

- When you give the command to execute, you do it like this: cat {}. So here, it is very beneficial to create a datatype for this that says the argument is either a flag or a path where you substitute in the actual filepath. And a complete command is just a list of these.

2.5 View Patterns

I made use of ViewPatterns language extension to keep my pattern matching concise. It lets me evaluate a function directly inside the pattern match, saving me from writing nested case expressions.

- Example:

```haskell
getExtentionFilter :: FilterFlags -> RawFilePath ->  Bool
getExtentionFilter sf fp = case extention sf of
                        Nothing    ->  True
                        (Just ext) -> case getFileExtention fp of
                                        Nothing     -> False
                                        (Just curr) -> curr == ext
```

- Becomes:

```haskell
getExtentionFilter :: Arguments -> RawFilePath -> Bool
getExtentionFilter (extention -> Nothing) _                                  = True
getExtentionFilter (extention -> Just _ )  (getFileExtention -> Nothing)   = False
getExtentionFilter (extention -> Just ext) (getFileExtention -> Just curr) = curr == ext
```

- I think this reads a lot better than the case statment, because it immediatly tells what output you get on that spesifc input. However this is just my subjektiv opinion.

2.6 A little note about FFI

Another thing i want to talk about i the calls done with FFI

```haskell
foreign import ccall safe "readdir"
  c_readdir :: Ptr CDir -> IO (Ptr CDirent)

foreign import ccall unsafe "__hscore_d_name"
  c_name :: Ptr CDirent -> IO CString

foreign import ccall unsafe "__posixdir_d_type"
  c_type :: Ptr CDirent -> IO DirType
```

- As you can see, c_readdir uses a safe call, but the other two use unsafe. Why is this? Normally, when we make a safe call with the FFI, the runtime releases the capability before doing any C stuff. This allows any other Haskell thread to grab the capability (basically the «right to run Haskell code»). This, of course, results in a bit of overhead. When we do an unsafe call, it skips all of this.

This can result in blocking the entire Haskell execution until C returns. So, if the C function takes a long time, or if it does not return, it can lead to a deadlock. It is also very important to use safe calls when the C code calls back into Haskell (not an issue here). So, it's important that we are selective with the functions we call unsafe with, and we should not use them for anything that can take a substantial amount of time (like networked file systems or slow spinning disks). Issue talking about this. This is why, when we do the readdir, we do it as a safe call. The other two are safe to make unsafe because they just read a field of a struct, which is very fast, and they do not do anything I/O related no syscalls, so there is no waiting.

3 Instructions

Check the README on gitlab for more info

```
cabal build

# eks.
cabal run lambdaSearch -- . -e hs
cabal run lambdaSearch -- -e hs

# For å kjøre testene
cabal test
```

4 Repo URL

https://git.app.uib.no/martin.e.knutsen/inf221-Semesteroppgave